

KOMYAKU — 多言語ドキュメント・バージョン管理プラットフォーム 設計仕様書

0. 文書情報

プロジェクト名：KOMYAKU / 稿脈

文書目的：Codexおよび開発者が、本仕様を基準としてKOMYAKUのMVPを設計・実装できる状態にする。

プロダクト定義：

KOMYAKUは、文章の完成状態だけではなく、文章がどのように生まれ、変更され、分岐し、採用・却下されたかという「稿脈」を保存・可視化する、多言語対応ドキュメント・バージョン管理プラットフォームである。

中心概念：

Document Evolution Graph

GitのCommit / Branch / DAGという優れた考え方を文書制作向けに再設計し、Gitそのものをユーザーに操作させない。

1. プロダクト原則

KOMYAKUは以下の原則を最優先する。

1.1 文書を失わない

入力中、保存中、同期中、サーバー障害時、誤操作時のいずれにおいても、可能な限りユーザーの文章を失わない設計とする。

1.2 過去を書き換えない

確定したVersionはImmutableとする。

Versionを変更するのではなく、必ず新しいVersionを生成する。

V1 → V2 → V3

V2を編集した場合もV2を書き換えず、

V1 → V2 → V3

V4

とする。

1.3 履歴そのものを第一級データとする

履歴はバックアップ機能ではない。

Document、Version、Version間の関係、変更理由、Branch、時刻、作者などをKOMYAKUの中心データとして扱う。

1.4 GitをUIに露出しない

内部思想として、

- Commit
- Branch
- DAG
- Merge
- Snapshot
- Content Hash

を参考にする。

ただしUIでは、

- Version → 版
- Branch → 別案
- Commit → 版を保存
- Checkout → この版を見る
- Revert → この版から復元
- Merge → 案を統合

など、人間の文章制作に即した表現を使用する。

1.5 ローカルファースト

ネットワーク障害が発生していても文章を書き続けられること。

クラウドとの通信成功を「入力保存」の前提条件にしない。

1.6 Internationalization First

多言語対応は後付け機能ではなく、データモデルの基礎要件とする。

UI言語と、ユーザーが文書内で使用する言語は完全に分離する。

2. 対応プラットフォーム

Tauri 2をアプリケーション基盤として使用する。

Tauri 2はWebフロントエンドとRust側ロジックを組み合わせ、macOS / Windows / Linux / iOS / Androidを対象にできる。

対象：

```
Web
macOS
Windows
Linux
iOS
Android
```

MVP優先順位：

```
1. Web
2. macOS
3. Windows
4. Linux
5. iOS
6. Android
```

ただしコード構造は初期段階から全プラットフォーム対応可能な状態とする。

3. 技術スタック

3.1 Frontend

```
Tauri 2
React
Vite
JavaScript
CSS
```

TypeScriptは必須としない。

UI：

```
React
CSS Variables
```

CSS logical properties
Intl APIs
i18next

i18nextはfallback languageを含む言語解決モデルを持つため、初期3言語から将来の言語追加へ拡張しやすい。

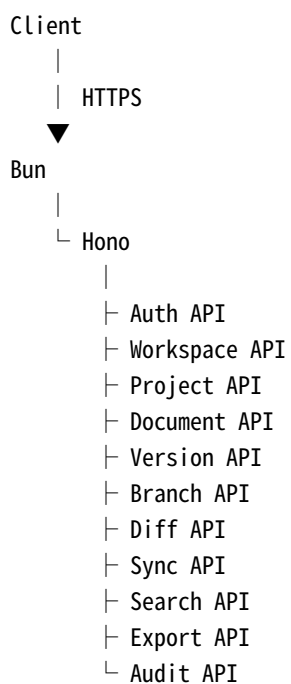
4. Backend

Bun
Hono
PostgreSQL
S3-compatible Object Storage

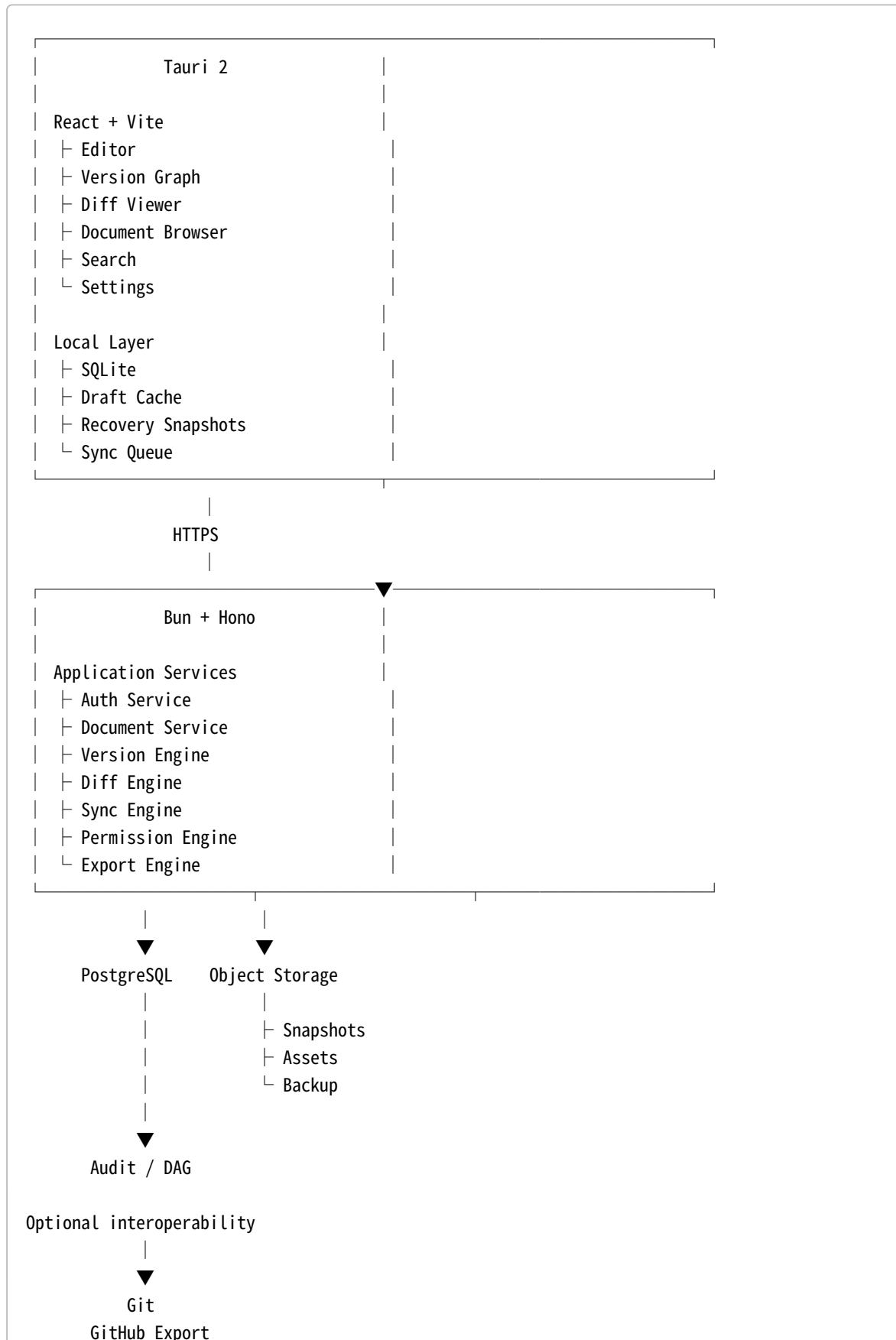
HonoはBunを正式に実行環境としてサポートしている。

BunにはPostgreSQLを含むSQLデータベース用APIが存在する。

構成：



5. 全体アーキテクチャ



Git Import
Git Sync

6. Gitの位置付け

GitをKOMYAKUの主データベースにしない。

KOMYAKU内部では独自Version DAGを実装する。

Gitは将来的な、

Import
Export
Backup
GitHub Sync
Interoperability

用途に限定する。

抽象インターフェイス：

VersionEngine

createVersion()
createBranch()
getVersion()
getHistory()
getAncestors()
getDescendants()
compareVersions()
restoreVersion()
mergeVersions()

将来的には、

VersionStore
└ PostgreSQLVersionStore
└ GitVersionStore

のように交換可能な設計とする。

7. データ階層

基本階層：

```
graph TD;
  User --> Workspace;
  Workspace --> Project;
  Project --> Document;
  Document --> Branch;
  Branch --> Version;
```

個人ユーザーであってもPersonal Workspaceを自動生成する。

これにより企業対応を後から追加してもデータモデルを変更しない。

8. Version DAG

VersionはNodeとして扱う。

Version間の親子関係をEdgeとして扱う。

```
graph TD;
  V1 --> V2;
  V2 --> V3;
  V3 --> V3_A[V3-A];
  V3 --> V4;
  V3 --> V3_B[V3-B];
  V4 --> V5;
```

MergeされたVersionは複数のParentを持てる。

```
graph LR;
  V5 --> V7;
  V6 --> V7;
```

そのため、

```
version_parents
```

を独立テーブルとする。

Versionテーブル内に単一の `parent_id` だけを持たせてはならない。

9. PostgreSQL基本スキーマ

最低限以下のEntityを作成する。

```
users
user_sessions

workspaces
workspace_members

projects

documents
document_members

branches

versions
version_parents

recovery_snapshots

assets

permissions
audit_logs

user_preferences
```

将来的に、

```
comments
reviews
merge_requests
semantic_changes
ai_operations
document_entities
```

を追加可能とする。

10. users

概念フィールド：

```
id
email
display_name

interface_locale
timezone

created_at
updated_at
deleted_at
```

`interface_locale` は固定Enumにしてはならない。

BCP 47 language tagを文字列として保持する。

例：

```
ja
ja-JP

en
en-US
en-GB

zh-Hans
zh-Hans-CN

ar
he
hi
th
```

11. UI Internationalization

初期UI対応言語：

```
ja
en
zh-Hans
```

つまり、

- 日本語
- English
- 简体中文

とする。

コード上にUI文字列を直接記述してはならない。

禁止：

```
<button>保存</button>
```

UIは必ずTranslation Keyを通す。

概念例：

```
t("document.save")
t("version.restore")
t("branch.create")
```

Translation Resource：

```
locales/
├─ ja/
│   ├─ common.json
│   ├─ editor.json
│   └─ version.json
├─ en/
│   ├─ common.json
│   ├─ editor.json
│   └─ version.json
└─ zh-Hans/
    ├─ common.json
    ├─ editor.json
    └─ version.json
```

言語を追加するとき、

```
locales/fr/  
locales/de/  
locales/ko/  
locales/ar/
```

を追加するだけで基本UIを拡張できる構造にする。

12. LocaleとLanguageを混同しない

以下は別データとして扱う。

```
interface_locale  
document_language  
content_language  
spellcheck_language  
writing_direction  
writing_mode
```

例：

日本人ユーザーが英語小説を書く場合：

```
interface_locale = ja  
document_language = en
```

英語UIを使用してアラビア語文書を書く場合：

```
interface_locale = en  
document_language = ar  
direction = rtl
```

13. 本文言語の基本方針

KOMYAKUは「サポート言語一覧」をハードコードしない。

文書言語には任意のBCP 47形式タグを設定可能とする。

例：

```
ja  
en
```

zh-Hans
zh-Hant
ko

ar
he
fa

hi
bn
ta
te

th
vi

ru
uk

el

fr
de
es

haw
mi

sa
la

存在しないタグやユーザー独自タグについても、保存自体を妨げない設計を検討する。

14. 「全言語対応」の定義

KOMYAKUでは「全言語」を、

Unicodeで表現可能であり、OSまたは利用環境に入力・表示手段が存在するテキストを、
KOMYAKU側が不必要に拒否・破壊・Latin文字前提処理しないこと

と定義する。

世界にはUnicode化されていない文字、未符号化文字、私用文字、歴史的表記なども存在し得るため、「地球上に存在するあらゆる言語を100%表示できる」と保証してはならない。

一方、KOMYAKU自身には特定言語Whitelistを持たせない。

15. Unicode

全システムの基本文字エンコーディング：

UTF-8

PostgreSQL Database EncodingもUTF8とする。

PostgreSQLはUTF8を含むmultibyte character encodingを扱える。

HTTP：

```
application/json; charset=utf-8
text/plain; charset=utf-8
text/html; charset=utf-8
```

Exportも原則UTF-8とする。

16. Unicode Normalization

ユーザーの原文を保存時に無条件でNFKCなどへ変換してはならない。

原稿そのもの：

original_content

は可能な限り入力された文字列を保持する。

検索・比較用には別途Normalization Viewを生成してよい。

推奨：

```
Storage:
exact authored Unicode content

Search projection:
NFC

Identifier/search special projection:
必要に応じて別途生成
```

UnicodeではNFC/NFD/NFKC/NFKDというNormalization Formが定義される。

特にNFKCは互換文字を統合するため、本文原稿を無条件NFKC変換しない。

17. Grapheme Cluster

KOMYAKUでは、

```
JavaScript string.length
```

を「文字数」として扱ってはならない。

ユーザーが認識する1文字とUnicode code point、UTF-16 code unitは一致しない場合がある。

Unicode Text Segmentationではgrapheme cluster、word、sentenceの境界が定義されている。

ブラウザ環境では可能な限り、

```
Intl.Segmenter
```

を使用する。

`Intl.Segmenter` はlocale-sensitiveなgrapheme / word / sentence segmentationを提供する。

Diff、文字数、カーソル単位などではgrapheme clusterを意識する。

18. 多言語Diff Engine

Diff Engineを「行単位Git Diff」のみに依存させてはならない。

最低限：

```
Paragraph Diff  
Sentence Diff  
Word Diff  
Grapheme Diff
```

のレイヤーを持てる設計とする。

日本語、中国語、タイ語などではスペース区切りを単語境界とみなしてはならない。

概念：

```
Document
↓
Block segmentation
↓
Sentence segmentation
↓
Word segmentation
↓
Grapheme segmentation
↓
Diff
```

Segmenterが利用できない場合はgrapheme-safe fallbackを設ける。

19. Bidirectional Text

LTRだけを前提としない。

対応方向：

```
ltr
rtl
auto
```

Unicode Bidirectional Algorithmを尊重する。

HTMLレンダリングでは必要に応じて、

```
lang
dir
```

属性を使用する。

W3CもRTL文書では基底方向を `dir="rtl"` 等で指定する方法を示している。

CSSでは、

```
margin-inline
padding-inline
inset-inline
border-inline
```

などlogical propertiesを優先し、

```
margin-left  
margin-right
```

依存を極力避ける。

RTL UIでも破綻しないレイアウトを目標とする。

20. Mixed Direction

一つの文書や段落内で、

```
Arabic + English  
Hebrew + numbers  
Japanese + English
```

が混在することを前提とする。

例：

```
للتوضيح 19 React هذا مثال
```

BiDi制御文字を不用意に削除・再配置しない。

Diff表示についてもLogical OrderとVisual Orderを混同しない。

21. 縦書き

MVPでは編集機能として必須にしない。

しかしデータモデルでは最初から、

```
writing_mode
```

を持てるようにする。

想定：

```
horizontal-tb  
vertical-rl  
vertical-lr
```


日本語組版では縦書きと横書きで文字、行、ページ進行方向などが異なる。

将来、

```
writing-mode: vertical-rl
```

を利用した縦書きViewer / Editorへ拡張できる構造とする。

22. Ruby / Annotation

日本語：

```
漢字
ふりがな
```

中国語：

```
汉字
拼音
```

などのannotationを後付けできるDocument Schemaを採用する。

本文中にHTML文字列を直接埋め込む方式是避ける。

概念：

```
text node
├─ annotations
│   └─ ruby
│       ├── base
│       └─ reading
```

23. Editor Document Model

文書本文を単なる巨大HTML文字列として保存しない。

内部Canonical Document Formatは構造化JSONとする。

推奨Editor Core：

ProseMirror系Document Model

ProseMirrorはSchemaによってdocument modelのnodeとmarkを定義する構造を持つ。

Canonical例：

```
Document
├ paragraph
│   ├── text
│   └── text
├ heading
├ blockquote
├ list
└ paragraph
```

ただしProseMirror固有JSONを永続フォーマットに直接固定しすぎない。

KOMYAKU独自のDocument Schema Versionを持つ。

```
schema_version
```

をすべてのVersion Snapshotへ保存する。

24. Document Schema Versioning

Editor Schemaは将来変更される。

したがって、

```
schema_version = 1
```

を各snapshotに保存する。

Schema変更時：

```
v1 document
↓
migration
↓
v2 document
```

とする。

過去Versionそのものは破壊しない。

必要に応じてViewer側でmigrationしたrepresentationを生成する。

25. 文書基本Node

MVP :

```
document  
  
paragraph  
heading  
text  
  
blockquote  
  
bullet_list  
ordered_list  
list_item  
  
hard_break  
horizontal_rule
```

Mark :

```
bold  
italic  
underline  
strike  
code  
link
```

Language metadata :

```
lang  
dir
```

をblockまたはinline rangeへ設定可能な拡張余地を持つ。

26. MVPで扱わない高度な組版

MVP対象外 :

本格DTP
多段組
ページ組版
脚注自動組版
数式組版
EPUB完全互換
Word完全互換
縦書き編集
複雑なruby
warichu
専門出版組版

ただしCanonical Schemaが後から拡張可能であること。

27. Local First Architecture

入力：

Keyboard / IME
↓
Editor Memory
↓
Local Draft
↓
Local SQLite
↓
Sync Queue
↓
Cloud

サーバー接続失敗で文章入力を止めない。

Local DBに最低限：

local_documents
local_drafts
local_snapshots
sync_queue
sync_state

を持つ。

28. IME

日本語、中国語、韓国語などのIME compositionを正しく扱う。

compositionstart

↓

compositionupdate

↓

compositionend

の途中状態を通常の確定Versionとして保存しない。

IME変換途中にDiffやAutosave処理がDOMを書き換えてcompositionを壊さないようにする。

29. Auto Save

3種類を分離する。

Draft State

非常に短い間隔。

ローカル中心。

Recovery Snapshot

数秒〜数十秒単位。

クラッシュ復旧目的。

Named Version

Version Graphに表示する意味のある版。

概念：

```
typing
↓
Draft
↓
Recovery
```

↓
Named Version

30. Version Graph

KOMYAKUの中心UI。

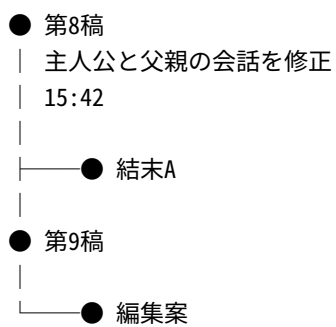
VS Code Git Graphに近い、



形式を採用する。

ただしコード開発者向け情報ではなく、文書向け情報を表示する。

例：



31. Version Graph Node

Version Nodeに表示可能な情報：

Version title

Author

Created time
Branch
Change summary

Added graphemes
Removed graphemes

Milestone status
Publication status

MVPでは：

title
time
branch
author

程度から開始する。

32. Graph操作

MVP：

Select Version
Compare
Restore
Create Alternative
Rename Version
Add Note

将来：

Merge
Review
Semantic History
Filter
Search Graph
AI analysis

33. Branch

内部：

```
branch
```

UI：

```
別案
```

BranchはVersionを複製するものではない。

Branchは、

```
head_version_id
```

を保持するlightweight referenceとする。

34. Version Snapshot

Version作成時には完全なDocument Snapshotを復元可能な形で保存する。

最低メタデータ：

```
version_id

document_id
branch_id

schema_version

created_at
created_by

title
note

content_hash
graph_hash
```

35. Content Hash

Version contentに対してcryptographic hashを生成する。

推奨：

SHA-256

目的：

corruption detection
integrity verification
deduplication assistance
export verification

Hashは認証そのものではない。

36. Graph Hash

Version自身のcontent hashとparent hash群からgraph hashを生成できる構造を設ける。

概念：

```
graph_hash =  
hash(  
    schema_version  
    content_hash  
    sorted(parent_graph_hashes)  
    immutable_metadata  
)
```

これによりVersion Graphの改ざん・破損検出能力を向上させる。

MVPでGraph HashをUI表示する必要はない。

37. Snapshot Storage

PostgreSQLのみを唯一の本文保存場所としない。

推奨：

PostgreSQL
metadata

Object Storage
immutable snapshots

PostgreSQL :

version graph
metadata
permissions
audit
indexes

Object Storage :

document snapshots
attachments
exports

38. Object Storage

S3 compatible APIを前提とする。

Object key例 :

workspaces/{workspaceId}/documents/{documentId}/versions/{versionId}.json

Object名に文書タイトルを使用しない。

ユーザー入力をStorage Pathへ直接使用しない。

39. Backup

最低3層 :

Primary Database
↓
PITR / Database Backup

Object Storage

↓
Object Versioning

Independent Backup
↓
Separate retention

バックアップを取るだけではなく、

Restore Test

を定期実施する。

40. Disaster Recovery

定期的に自動復元試験を実施可能な設計とする。

例：

Random Backup Sample
↓
Restore
↓
Hash Verification
↓
Version DAG Reconstruction
↓
Document Rendering
↓
Success / Alert

41. Export

ユーザーのデータをKOMYAKUにロックインしない。

必須Export：

Plain Text
Markdown
JSON
KOMYAKU Archive

将来：

```
DOCX
PDF
EPUB
Git Repository
```

42. KOMYAKU Archive

独自Archive例：

```
novel.komyaku
```

実体は標準的なArchive形式を使用する。

内部：

```
manifest.json

documents/
  document.json

versions/
  {version-id}.json

graph/
  graph.json

assets/
```

KOMYAKUサービスが存在しなくなっても、ユーザーがVersion Historyを復元できる構造を目標とする。

43. Authentication

MVP：

```
Email
Password
Session
Email verification
Password reset
```

設計段階から将来：

Passkey
OAuth
MFA
OIDC
SAML
SCIM

を追加可能にする。

Passwordを独自暗号方式で実装しない。

44. Session Management

Sessionをユーザー単位で管理する。

将来UI：

Current device

MacBook
iPhone
Chrome Windows

操作：

Revoke session
Logout all devices

45. Workspace

全Userは少なくとも一つのWorkspaceに属する。

個人登録時：

User
↓
Personal Workspace

を自動作成する。

46. Permission Model

内部モデル：

```
Owner
Admin
Editor
Reviewer
Viewer
```

MVP UIでは主にOwnerのみ使用してよい。

しかしDatabase Schemaを単一所有者前提にしてはならない。

47. Audit Log

重要操作をAudit Logへ記録する。

```
actor
action
resource_type
resource_id

timestamp

ip metadata
device metadata
```

例：

```
DOCUMENT_CREATED
VERSION_CREATED
VERSION_RESTORED
BRANCH_CREATED
DOCUMENT_DELETED
DOCUMENT_EXPORTED
LOGIN
LOGOUT
```

Audit Log自体を通常ユーザー操作で編集できないようにする。

48. Soft Delete

重要データは即時物理削除しない。

```
active
↓
trashed
↓
retention
↓
purged
```

Document削除：

まずTrashへ移動する。

Versionについても参照整合性を保つ。

49. Security Boundary

Workspace境界を必ずBackendで検証する。

FrontendのUI非表示をPermission enforcementとして扱ってはならない。

すべてのAPIで：

```
Authenticated User
↓
Workspace Membership
↓
Resource Permission
↓
Operation
```

を検証する。

50. API設計

Base：

```
/api/v1/
```

例：

```
POST    /auth/login
POST    /auth/logout

GET     /workspaces
GET     /projects

GET     /documents
POST    /documents

GET     /documents/:id

GET     /documents/:id/versions
POST    /documents/:id/versions

GET     /versions/:id

GET     /documents/:id/branches
POST    /documents/:id/branches

POST    /versions/:id/restore

POST    /diff

POST    /sync
```

51. API Versioning

APIの破壊的変更に備える。

```
/api/v1
/api/v2
```

とする。

Document Schema VersionとAPI Versionは別物。

52. ID

外部公開EntityのIDには連番integerを使用しない。

推奨：

UUIDv7

または同等のsortable unique identifier。

DB内部最適化のため別surrogate keyを使用する余地は残す。

53. Time

Database内timestamp：

UTC

ユーザー表示：

user timezone

APIではISO 8601互換形式を使用。

UI表示は `Intl.DateTimeFormat` 等のlocale-aware APIを使用する。

54. 数値・日時・複数形

翻訳JSONに、

"3 versions"

のような組み立て文字列を直接作らない。

Locale-aware formattingを使用する。

対象：

Date
Time
Number
Plural
Relative Time
List

UI言語追加時にもコード変更を最小化する。

55. Search

MVP :

```
document title
current document body
```

将来 :

```
all versions
deleted text
version notes
semantic search
```

へ拡張する。

検索処理でLatin alphabet前提のtokenizationをしない。

56. Search Normalization

検索Index用文字列と原文を分離する。

```
Original
↓
Search Projection
↓
Index
```

Search Projectionの変更によってOriginal Documentを変更してはならない。

57. Document Metadata

基本 :

```
id
workspace_id
project_id

title
```

```
default_language  
default_direction  
default_writing_mode
```

```
created_by  
created_at  
updated_at
```

```
current_branch_id  
current_version_id
```

58. 文書内複数言語

Documentにはdefault languageを持たせる。

しかしblock単位・inline単位でoverride可能なSchemaにする。

例：

```
Document: ja
```

```
Paragraph 1: ja
```

```
Paragraph 2: en
```

```
Paragraph 3:
```

```
  ja
```

```
  inline Arabic: ar / rtl
```

これにより翻訳文、語学教材、学術文書なども扱える。

59. Attachments

MVPでは画像程度までを想定してよい。

AttachmentはDocument JSONへBase64埋め込みしない。

Object Storageへ保存し、

```
asset_id
```

で参照する。

60. Diff Storage

DiffをVersionの唯一の保存形式にしない。

Canonical VersionはSnapshot。

Diffは、

```
computed artifact  
cache
```

として扱う。

Versionを復元するために全Diffを順番に適用しなければならない設計を避ける。

61. Recovery Snapshot

Recovery SnapshotとNamed Versionを別テーブルまたは明確に別種別として扱う。

Recovery :

```
short-lived  
automatic  
high frequency
```

Named Version :

```
immutable  
long-term  
graph-visible  
user meaningful
```

62. Sync

同期単位 :

```
document_id  
base_version_id  
local_revision
```

Network切断中の編集を許可する。

MVPでは複数端末で同時編集された場合に完全自動Mergeを要求しない。

競合時：

```
Device A Version
Device B Version
```

としてBranchを自動生成してもよい。

文章を消すより分岐させることを優先する。

63. Conflict Philosophy

KOMYAKUでは、

Conflict is a branch, not data loss.

を基本原則とする。

競合が解消できない場合：

```
Main
├ Device A changes
└ Device B changes
```

として両方を保存する。

64. MVP UI

最低限4画面。

Library

```
Workspace
Project
Documents
```

Editor

Document
Editor
Save Version

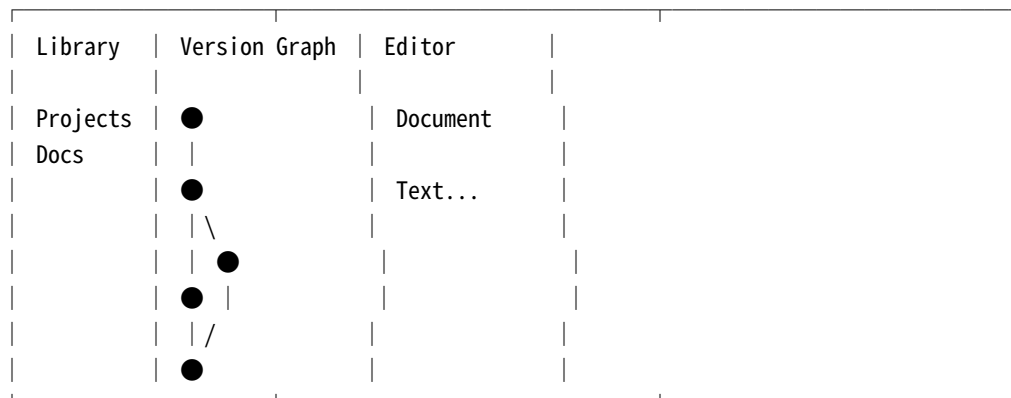
Version Graph

Graph
Version metadata

Diff Viewer

Old
New
Changes

65. Desktop基本レイアウト



GraphをKOMYAKUの視覚的アイデンティティとする。

66. Responsive

Mobileでは3ペインをそのまま縮小しない。

Library
↓
Document

↓
Editor
↓
History

のNavigationに変換する。

67. Accessibility

最低限：

Keyboard navigation
Semantic HTML
ARIA where necessary
Focus visibility
Screen reader labels
Reduced motion
High contrast compatibility

Graph情報を色だけで表現しない。

Node shape、line、labelなども併用する。

68. UI方向

スペーシー、SF的、過剰なGlass UIにはしない。

文書制作を邪魔しない落ち着いたインターフェイスとする。

中心は文章。

Graphは機能的な可視化とする。

69. MVP必須機能

実装する：

User registration

Login
Logout

Personal Workspace

Project

Document create

Document edit

Local autosave

Cloud sync

Named Version create

Immutable Version

Version DAG

Version Graph

Alternative branch create

Version compare

Grapheme-safe Diff

Restore from version

Trash

Backup

Basic Export

UI:

ja

en

zh-Hans

70. MVPから外すもの

実装しない：

AI writing

Semantic Diff

AI history search

Realtime collaboration

Comments
Reviews

Complex RBAC UI

Merge UI

SSO
SAML
SCIM

EPUB

Advanced DOCX

Professional PDF typesetting

Vertical writing editor

Publishing platform

Social network

Plugin ecosystem

Marketplace

71. ただし将来対応を妨げてはならないもの

データ構造上は以下を想定する。

Collaboration
AI
Semantic metadata

Publishing
Enterprise

Vertical writing

Ruby

Math

Comments

Review

Merge

Translation workflow

Audio/video attachment

72. Phase 2

MVP後：

Semantic Diff

AI change summary

Graph search

Version filtering

Document-wide search

Vertical Japanese viewer

Ruby

DOCX Import/Export

Git Export

73. Phase 3

Collaboration

Comments

Review

Merge

Publisher workflow

Advanced permissions

74. Phase 4

Enterprise

SSO

SAML

SCIM

Compliance features

Organization management

API

GitHub integration

AI semantic history

75. AI将来構想

AIは文章生成よりもVersion History理解を優先する。

例：

「主人公の職業を変更したのはいつ？」

「削除した伏線を探して」

「この2つのBranchの違いを説明して」

「第3稿から主人公の性格はどう変化した？」

「以前却下した案の中から、この問題を解決できるものを探して」

Version GraphそのものをAI Contextとして利用する。

76. Semantic Version Metadata

将来的にVersionへ、

```
people
places
dates
relationships
plot changes
tone changes
terminology changes
```

などを保存可能にする。

ただしMVPのVersion SchemaをAI依存にしない。

77. Git interoperability

将来的なGit Export :

```
KOMYAKU Version
↓
Git Commit

KOMYAKU Branch
↓
Git Branch
```

KOMYAKUのVersion DAGとGit Graphの意味対応を維持する。

ただしGitの内部制約をKOMYAKU DB Schemaへそのままコピーしない。

78. Repository構成

推奨Monorepo :

```
komyaku/
|
├─ apps/
|   ├─ desktop/
|   └─ server/
|
```

```
└ packages/
  │   └ editor-core/
  │   └ document-schema/
  │   └ version-engine/
  │   └ diff-engine/
  │   └ sync-core/
  │   └ i18n/
  │   └ api-client/
  │   └ shared/
  │
  └ database/
    │   └ migrations/
    │   └ seeds/
    │
    └ locales/
      │   └ ja/
      │   └ en/
      │   └ zh-Hans/
      │
      └ docs/
        │   └ architecture/
        │   └ schema/
        │   └ adr/
        │
        └ README.md
```

79. Tauri構成

```
apps/desktop/
├
└ src/
  │   └ app/
  │   └ components/
  │   └ editor/
  │   └ graph/
  │   └ diff/
  │   └ library/
  │   └ hooks/
  │   └ services/
  │   └ i18n/
  │
  └ src-tauri/
    │   └ src/
    │   │   └ commands/
    │   │   └ local_db/
    │   │   └ filesystem/
    │   │   └ sync/
    │   │   └ security/
```

```
|   └─ tauri.conf.json
|
|   └─ vite.config.js
```

TauriではWeb frontendとRust側application logicを分離できる。

80. Server構成

```
apps/server/
|
├─ src/
|   └─ index.js
|   |
|   └─ routes/
|       └─ auth.js
|       └─ documents.js
|       └─ versions.js
|       └─ branches.js
|       └─ sync.js
|       |
|       └─ services/
|           └─ auth/
|           └─ document/
|           └─ version/
|           └─ diff/
|           └─ storage/
|           └─ sync/
|           |
|           └─ middleware/
|           └─ repositories/
|           └─ security/
|           └─ config/
|
└─ package.json
```

81. Domain Layerを分離する

Route handler内へVersion処理を直接実装しない。

禁止：

```
HTTP Route
↓
SQL
```

↓
Version creation

推奨：

Route
↓
Application Service
↓
Domain Service
↓
Repository
↓
Database

Version EngineをHonoから独立させる。

82. Version作成Transaction

Named Version作成時：

Validate Permission
↓
Validate Document
↓
Serialize canonical snapshot
↓
Calculate content hash
↓
Write immutable object
↓
Create Version row
↓
Create parent edges
↓
Update branch head
↓
Audit log
↓
Commit transaction

途中失敗で半端なVersionを公開しない。

83. Storage Failure

Object Storageへのsnapshot保存が失敗した場合、

Version metadataだけを確定しない。

Transactional OutboxまたはPending Stateを利用できる設計を検討する。

Versionは、

```
pending
ready
failed
```

等の内部状態を持てる。

UIに表示するVersionは原則 `ready` のみ。

84. Concurrency

Branch head更新には競合検出を入れる。

例：

Client：

```
expected_head = V10
```

Server：

```
current_head = V11
```

なら無条件上書きしない。

新Branch生成またはConflict responseにする。

85. Database Constraints

Application logicだけに頼らない。

最低限：

FOREIGN KEY
UNIQUE
NOT NULL
CHECK

を適切に使用する。

Workspace境界等はRepository Layerでも検証する。

86. テスト戦略

最低限：

Unit Test
Integration Test
Database Test
Version DAG Test
Unicode Test
Diff Test
Sync Test
Recovery Test
E2E Test

87. Unicode Test Corpus

最低限以下をテストする。

Japanese
English
Simplified Chinese
Traditional Chinese
Korean

Arabic
Hebrew
Persian

Hindi / Devanagari
Bengali
Tamil

Thai

Vietnamese

Cyrillic

Greek

Emoji

Combining characters

Variation selectors

ZWJ emoji

Mixed RTL/LTR

88. 特に壊してはいけない文字列

テストには以下の種類を必ず含める。

é

e + combining acute



漢字

العربية

עברית

हिन्दी

ภาษาไทย

目視だけでなく、

UTF-8 round trip

hash

snapshot

restore

diff

まで検証する。

89. Round Trip Test

重要テスト：

```
Input
↓
Local save
↓
API
↓
PostgreSQL/Object Storage
↓
Reload
↓
Export
↓
Import
↓
Compare
```

最初の文字列と最後の文字列が意図せず変化していないことを確認する。

90. Version Property Test

任意Version Vについて：

```
restore(V)
```

した結果が、V作成時snapshotと一致すること。

任意Graphについて：

```
parents(V)
```

が失われないこと。

Branchを削除してもVersionが参照条件に従って保全されること。

91. Performance初期目標

MVPで想定する通常文書：

1 document:
数百KB～数MB

versions:
数百～数千

Graph全件をDOM nodeとして一度に描画する設計を避ける。

必要になればVirtualizationを導入する。

92. 巨大文書

長期的に：

novel
research notebook
manual
corporate documentation

など数十年使われる可能性を考える。

「Documentは常に小さい」と仮定しない。

Document segmentation / chapter structureを後から導入できるID設計とする。

93. 観測性

Server：

structured logs
request ID
trace ID

latency
error rate

database latency
storage latency

sync failure
version failure

を観測可能にする。

本文そのものを通常application logへ出力してはならない。

94. Privacy

本文、Version、Diffはユーザーの機密情報になり得る。

原則：

Do not log document content.
Do not log passwords.
Do not expose raw snapshots through predictable URLs.

Object Storageへのアクセスは認証済みAPIまたは短時間signed accessを通す。

95. MVP成功条件

ユーザーが次の操作を迷わず行えること。

文章を書く
↓
版を保存
↓
さらに書き直す
↓
別案を作る
↓
Graphを見る
↓
過去版を選ぶ
↓
現在版と比較
↓
過去版から復元

さらに、

アプリを強制終了

↓

再起動

↓

直前の文章が復旧

すること。

96. KOMYAKUの中心的Invariant

実装中、以下を絶対条件として扱う。

Invariant 1

確定Versionは変更しない。

Invariant 2

Versionは最低1つの完全な復元手段を持つ。

Invariant 3

Version GraphのEdgeを勝手に変更しない。

Invariant 4

同期競合で本文を捨てない。

Invariant 5

UI LocaleとDocument Languageを分離する。

Invariant 6

Latin/LTR/space-separated languageを前提としない。

Invariant 7

JavaScript `string.length` を人間の「文字数」とみなさない。

Invariant 8

本文を無条件Unicode compatibility normalizationしない。

Invariant 9

Cloud接続失敗で入力を失わせない。

Invariant 10

KOMYAKUからユーザーデータをExportできるようにする。

97. Codexへの実装指示

最初から全機能を実装しないこと。

以下の順で実装する。

Stage 1 — Foundation

```
Monorepo
React/Vite/Tauri
Bun/Hono
PostgreSQL

config
migration
logging
i18n

ja
en
zh-Hans
```

Stage 2 — Identity

```
User
Session
Personal Workspace
Project
```

Stage 3 — Document

```
Document Schema
Editor
Local Draft
Cloud Save
```

Stage 4 — Version Engine

Version
Version Parents
Branch
Immutable Snapshot
Hash

Stage 5 — Version Graph

Graph rendering
Version selection
Branch visualization

Stage 6 — Diff

Unicode segmentation
Paragraph Diff
Word/Grapheme Diff
Diff UI

Stage 7 — Recovery

Recovery Snapshot
Offline
Sync Queue
Conflict Branch

Stage 8 — Durability

Object Storage
Backup
Export
Restore tests

98. Codexへの禁止事項

以下を行わないこと。

Git repositoryを主DBにしない

HTML文字列だけをcanonical documentにしない

VersionをUPDATEしない

UI文字列をcomponentへ直書きしない

languageをenum固定しない

document languageとinterface localeを同一フィールドにしない

RTLを後回しにする前提のCSSを書かない

space splitでword diffを作らない

string.lengthで文字数を数えない

autosaveだけをversion historyとして扱わない

本文をapplication logsへ書かない

document titleをstorage pathとして使わない

同期Conflict時にlast-write-winsで片方を消さない

PostgreSQLだけを唯一のVersion Snapshot保存先にしない

MVPへAIを追加しない

MVPへRealtime collaborationを追加しない

99. Architecture Decision Records

重要な判断は、

docs/adr/

へADRとして保存する。

最低限：

ADR-001 Version DAG

ADR-002 Canonical Document Format

ADR-003 Unicode Policy

ADR-004 Local First Sync
ADR-005 Snapshot Storage
ADR-006 Internationalization
ADR-007 Version Immutability
ADR-008 Git Interoperability

を作成する。

KOMYAKU自身が履歴を重要視する製品なので、KOMYAKU開発そのものについても設計判断履歴を残す。

100. 最終プロダクト思想

KOMYAKUは、

「文書を保存するサービス」

ではない。

目指すものは、

文書が生まれ、変化していった時間構造を保存する基盤

である。

完成稿だけではなく、

Draft
↓
Revision
↓
Alternative
↓
Rejected Idea
↓
Reconsideration
↓
Merge
↓
Published Version

という思考の歴史を残す。

そしてその履歴が、

Today
Tomorrow

1 year later
10 years later

でも復元可能であることを重視する。

KOMYAKUの中心価値は、

Write

Evolve

Remember

である。

日本語コピー：

書いた文章だけでなく、書き直した道のりも残す。

設計上の最重要目標：

Language-independent, local-first, immutable document evolution infrastructure.

KOMYAKUを単なる日本語執筆エディタとしてではなく、

長期間使い続けられる、人間の文書と思考履歴の基盤

として設計すること。